

## 8. Proposed System

---

### 8.1 Overview

The system proposed in this work is a web application that estimates a person's financial risk profile from a set of self-reported demographic, financial and lifestyle attributes, and then explains how it arrived at that estimate. It was built as three cooperating layers: a browser client, a REST service, and a machine learning module that the service calls in-process.

The design goal was narrow and deliberate. Rather than attempting to build a general-purpose financial advisory tool, the system answers one question well: given twenty-six inputs describing a person's money habits, which of three risk bands do they fall into, how confident is that answer, and which of their inputs pushed the answer in that direction. Everything else in the application exists to support that question or to make its answer legible.

### 8.2 What the System Does

A user opens the application and works through a four-step form. The first step asks for demographic details such as age, education and employment type. The second covers the financial picture — income, expenses, existing debt, credit score, how long they have been investing. The third asks about lifestyle spending: how often they shop, what fraction of their expenses go online, how many subscriptions they hold. The fourth step simply shows everything back for review before submission.

When the form is submitted, the browser sends a single JSON document to the backend. The backend validates it, hands it to the machine learning module, receives a prediction along with the per-feature attributions that produced it, writes the whole exchange to a relational database, and returns the result. The browser then renders the outcome: a risk band, a confidence figure, three behavioural scores, a ranked chart of which features mattered, a list of behavioural biases detected by rule, and a set of written recommendations.

Nothing is computed in the browser. The frontend holds no thresholds, no scoring formulas and no model. This was a conscious separation. If the model is retrained or a scoring rule changes, no frontend code needs to be touched, and there is exactly one place where the behaviour of the system is defined.

### 8.3 Distinguishing Characteristics

**A single definition of feature engineering.** An early version of this system computed derived features twice — once in the offline preprocessing script that prepared the training data, and again inside the inference module that served live requests. The two implementations disagreed on five separate points, and because the offline test set was evaluated using the offline code path, the accuracy figures never revealed the problem. Section 22 documents this in detail. The proposed system eliminates the possibility: the preprocessing script writes every constant it fits — ordinal encodings, outlier bounds, normalisation ranges, imputation medians — to a JSON artefact, and the inference module reads that artefact rather than restating the values. The training and serving paths are now verified to agree exactly.

**Explanations are produced per prediction, not per model.** A global feature-importance ranking tells the reader which features the model relies on in aggregate. It does not tell an individual user why *their* assessment came out the way it did. The system therefore computes SHAP values for each request, so that the chart a user sees reflects the contribution of their own inputs.

**Predictions are stored, not just returned.** Each assessment persists the submitted inputs, the resulting prediction, the SHAP attributions and the derived scores. This supports the history and analytics views, and it means a prediction can be reproduced and audited after the fact.

**The system reports its own uncertainty.** The classifier returns a probability across all three risk bands rather than a bare label, and the interface displays the probability of the predicted class. Where the model is unsure, the interface shows that it is unsure.

## 8.4 Scope and Boundaries

Two boundaries are worth stating plainly at the outset, because they shape how the results in Section 22 should be read.

First, the data underlying the trained model is synthetic. No survey was conducted and no human respondent contributed a row. The dataset was produced by a generative script that samples correlated demographic, financial and lifestyle attributes and then derives the target labels from a documented scoring function. This is a legitimate methodology for a system whose purpose is to demonstrate an end-to-end pipeline, and it is used here because no public dataset combines these particular twenty-six attributes with a behavioural risk label. It does mean, however, that the reported accuracy measures how well the model recovers a known generating process, not how well it would predict the risk profile of a real person. Section 12 describes the generator; Section 26 returns to the consequences.

Second, the system predicts a risk *category*. It does not give financial advice, and the recommendations it produces are generated by fixed rules over the user's inputs rather than by the model. They are illustrative rather than prescriptive.

## 8.5 Comparison with the Existing Approach

Conventional risk profiling in retail finance relies on a questionnaire whose answers are mapped to a score by a fixed, hand-written rubric. Each answer carries a preset weight; the weights are summed; the total is compared against cut-offs. The approach is transparent and cheap, and it is easy to defend to a regulator, which is largely why it persists.

Its weaknesses are equally well known. The weights encode an analyst's assumptions rather than any observed relationship in data. Interactions between attributes are ignored, so a rubric cannot express the idea that a low savings rate matters more for someone with dependants than for someone without. And because the rubric is fixed, it cannot improve as data accumulates.

The system proposed here replaces the rubric with a learned function while retaining the property that made the rubric attractive. A tree ensemble captures interactions and non-linear thresholds that a linear rubric cannot represent, and SHAP restores the per-answer attribution that a rubric gives away for free. The user still sees which of their answers moved the outcome and by how much; the difference is that the magnitudes are learned rather than assumed.

**Figure 8.1** — *System context diagram*. Show the three actors (user's browser, REST service, ML module) and the database, with arrows labelled by what crosses each boundary (JSON request, feature vector, prediction + SHAP, persisted row). Place after Section 8.2. Source provided in Section 11.

**Table 8.1** — *Comparison of rule-based rubric scoring against the proposed learned approach*. Rows: basis of weights, handling of feature interactions, per-user explanation, ability to improve with data, auditability. Place at the end of Section 8.5.